2025 秋计算思维与实践期末考试（回忆版）一二三大题答案

回忆:（基石铸魂讲师团）ztj，红笺小字，航

声明：答案未精校，包含部分AI辅助生成内容，不代表任何官方组织或教师，内容需要自行分辨，仅供参考。

一、选择题

1. (d) 系统总线

2. (c) 21

3. (c) 1121

4. (c) 444

5. (b) 1331

6. (d) *(arr+1)+2

7. (a) 带头尾指针的双向链表

8. (d) D, C, E, A, B

9. bc

二、基础填空题

1. 补码

2. 5.5（计算：5/2=2，2+1.5+2.0=5.5）

3. 入口地址（首地址/内存地址）

4. malloc：int (*arr)[4] = (int(*)[4])malloc(5 * sizeof(int[4]));
   calloc：int (*arr)[4] = (int(*)[4])calloc(5, sizeof(int[4]));

三、程序填空题

1. 计算字符串长度的 My_strlen 函数

```
#include <stdio.h>
int My_strlen(char *p)
{
    int len = 0;
    while(*p != '\0')  // 第一个填空：*p != '\0'
    {
        len++;
        p++;            // 第二个填空：p++
```

```
    }
    return len;
}
int main()
{
    char str[] = "Hello";
    int length = My_strlen(str);  // 第三个填空：str
    printf("字符串 '%s' 的长度是:%d\n", str, length);
    return 0;
}
```

2. 计算整数 n 各位数字之和的递归函数

```
int sum(int n) {
    if (n < 10) {
        return n;  // 第一个填空：n
    } else {
        return n%10 + sum(n / 10);  // 第二个填空：n%10
    }
}
```

四、简答与分析题

1.

(a)

1. **顺序存储（数组实现）**
   - **用一组地址连续的存储单元依次存储线性表的数据元素。**
   - **逻辑上相邻的元素在物理位置上也相邻。**
   - **优点：随机存取，查找方便。**
   - **缺点：插入删除需要移动大量元素。**
2. **链式存储（链表实现）**
   - **用一组任意的存储单元存储元素，通过指针链接元素。**
   - **逻辑上相邻，物理上不一定相邻。**
   - **优点：插入删除不需要移动元素，只需修改指针。**
   - **缺点：需要额外存储指针，不能随机存取。**

注：答出顺序存储、链式存储即可给分；答栈和队列需酌情扣分。

(b)

- **栈**只能在**一端（栈顶）**进行插入（push）和删除（pop）操作，即后进先出（LIFO）。
- **普通数组**可以在任意位置进行插入和删除操作，并且元素可以任意访问（不限于一端）。

本质区别：

- **操作限制**：栈是一种**受限的线性表**，只允许在一端操作。
- **普通数组**（作为线性表）对操作位置没有限制（只要实现支持，可任意位置插入删除）。

注：答出“栈只能在栈顶”“数组任意位置”，大意对即可给分。

(c)

动态内存分配：链表实现栈不会像数组那样预先分配固定容量，可以按需分配，避免了栈满（固定大小数组）的问题。

- **插入删除效率**：入栈出栈只需修改头指针，不需要移动大量元素（但数组也能 O(1) 压栈如果容量足够且只在尾部操作，但扩容时需复制）。
- 实际实现中，链表栈的每个节点单独分配，不会浪费连续内存空间。

注：动态分配/防满 是一点，效率高是一点，大意对即可给分，漏点需酌情扣分。

(d)

需要支持：

1. **相邻切换**（前后切换）：需要知道当前页的前一个和后一个，所以结构应该能向前/向后访问相邻元素。
2. **拖拽调整顺序**：需要能方便地把一个标签移到另一个位置（插入删除）。

分析：

- **数组**：随机访问快，但拖拽调整顺序需要移动大量元素，效率低。
- **栈**：不能随意调整中间顺序，不能前后切换（除非用两个栈，但复杂且不适合）。
- **单向链表**：只能向后访问，不能向前找前一个，不适合前后切换。
- **双向链表**：
  - 每个节点有 prev 和 next 指针，前后切换（O(1)）。
  - 拖拽调整顺序（删除节点 + 插入到某节点前后）只需修改几个指针，效率高。

结论：最适合的是 **双向链表**。

注：答出“双向链表”，并简要说明便于前后切换、调顺序效率高的优点，大意对即可给分。

2.

选择排序基本思想：

每一趟在待排序元素中选出最小（或最大）的一个元素，与序列起始位置交换，直到全部排完。

直接插入排序基本思想：

将待排序元素插入到已经有序的部分序列中的正确位置。

注：基本思想大意对即给分，若用作图等方式辅助文字说明，意思对即可给分。

序列 {15, 12, 10, 38, 45} **第一趟排序结果：**

- 选择排序：找到最小元素 10，与第一个元素 15 交换 → **{10, 12, 15, 38, 45}**。
  注：结论对即给分。若前面基本思想选的是最大而不是最小，此处可以是 45，12，10，38，15，亦给分。
- 直接插入排序：前 1 个元素 [15] 有序，插入 12：12 < 15，放在 15 前面 → **{12, 15, 10, 38, 45}**。
  注：顺序结论对即给分。

算法分析：

- **选择排序：**
  - 平均时间复杂度 $O(n^2)$
  - 平均空间复杂度 $O(1)$
  - 稳定性：不稳定（例：5 8 5 2，第一个 5 与 2 交换后相对次序变化）
- **直接插入排序：**
  - 平均时间复杂度 $O(n^2)$
  - 平均空间复杂度 $O(1)$
  - 稳定性：稳定（相等元素不交换位置）

注：二者相同项一起写也给分；结论均正确即给分，未举例分析不扣分。

3.

两个主要优点：

1. **避免假溢出：**当 rear 到达数组末尾时，可以绕回到开头继续使用空闲位置，提高空间利用率。
2. **插入和删除操作时间复杂度 O(1)**，且不需要搬移元素，比普通队列（若用数组实现）更高效。

注：大意对即给分。

判空与判满条件：

- 队空条件：front == rear
- 队满条件：(rear + 1) % maxsize == front

4.

- 方案 A：

text

```
p->next = k;
k->next = p->next;
```

先修改 p->next 指向 k，再让 k->next 指向 p->next（即 k 自己），造成错误。

- 方案 B：

text

```
k->next = p->next;
p->next = k;
```

这是正确的，先保存 p 原来的后继到 k->next，再让 p->next 指向 k。

(a) 正确的是 **方案 B**。
注：写出 B 对即可给分。

(b) 若用方案 A：

- 后果：
  - 首先 p->next = k，丢失原 p 的后继节点（无法找回）。
  - 然后 k->next = p->next，此时 p->next 已变成 k，所以 k->next = k，形成自环。
  - 链表断链，原 p 之后节点丢失，链表结构破坏。

注：大意正确即可给分。

5.

(a) 能实现删除效果吗？为什么？

- 可以，这条语句执行后，k 直接跳过原 k->next 指向原 k->next 的下一个节点。
- 原被删节点从链表逻辑上断开，但在内存中还存在（内存泄漏）。

注：大致答出能实现删除即可给分。

(b) 缺点与后果：

- **内存泄漏**：没有释放被删除节点的内存（用 free 或 delete）。
- 如果 k->next 为 NULL，则 k->next->next 访问空指针，程序崩溃。
- 无法在删除后获取被删除节点的值（指针丢失）。
- 不符合安全编程规范。

注：内存未清空是一点，访问空指针是一点，大致意思对即可给分。漏点酌情扣分。

五题答案

红笺小字 2026/2/28

题目

1. 给定一个已按从小到大排序的整型数组 numbers[]，数组大小为 N，数组索引*从 1 开始计数*。请找到数组中两个不同位置的数，使它们的和等于目标值target。题目保证若解存在则唯一。请将找到的两个下标index1 和index2 存入数组 arr[] 中，其中 arr[0] = index1，arr[1] = index2，且满足 index1 < index2。

请完成以下任务：
(a) 请写出完整的函数原型。
(b) 在第 1 问的基础上，使用二分查找的思路，用C语言实现该函数。
(c) 写出第 1 问和第2问实现的算法的时间复杂度和空间复杂度（计算时忽略存储结果数组arr 本身的大小）

(a) 请写出完整的函数原型

方法一：暴力算法

思路与算法

枚举数组中每一个数`x`，寻找数组中是否存在`target - x`
使用遍历整个数组方式寻找`target - x`，需要注意每一个之前的元素都已经匹配过，不需要再进行匹配
顺便说一下，可以直接去leetcode第一题找题解

```c
int* twoSum(int *numbers, int N, int target) {
    for (int i=0;i<N-1;i++)
    {
        for (int j=i+1;j<N;j++)
        {
            if (numbers[i] + numbers[j] == target)
            {
                int* arr = malloc(sizeof(int)*2);
                arr[0] = i + 1; //转换为1开始计数
                arr[1] = j + 1;
                return arr;
            }
        }
    }
    return NULL;
}
```

时间复杂度： $O(N^2)$ 空间复杂度： $O(1)$

(b) 在第 1 问的基础上，使用二分查找的思路，用C语言实现该函数。

方法二：二分法

思路与算法

同样固定一个数，二分查找另一个数，注意二分查找方法即可，注意`mid+1`与`mid-1`

```c
int* twoSum(int *numbers, int N, int target) {
    for (int i=0;i<N;i++)
    {
        int left = i+1;
        int right = N-1;
        int complement = target - numbers[i];
        while (left <= right)
        {
            int mid = (right - left)/2 + left; //防止整数溢出
            if (numbers[mid] == complement)
            {
                int* arr = malloc(sizeof(int)*2);
                arr[0] = i+1;
                arr[1] = mid+1;
                return arr;
            }
            if (numbers[mid] < complement)
            {
                left = mid + 1;
            }
            else
            {
                right = mid - 1;
            }
        }
    }
    return NULL;
}
```

时间复杂度：$O(Nlog(N))$ 空间复杂度：$O(1)$

---

2. 假设你正在爬一段有n阶台阶的楼梯，从第一阶开始爬，每次只能爬1阶或2阶。请设计递归函数ClimbStairs，计算爬到楼顶有多少种不同的方法。
   请回答以下问题：
   ***(a)** 什么是递归？一个递归函数必须包含哪两个核心部分？它们各自的作用是什么？*
   ***(b)** 根据问题描述，写出计算**ClimbStairs(n)** 的递归表达式（需包含基准条件和递归条件）。*
   ***(c)** 请用 **C** 语言编写完整的程序，实现**ClimbStairs** 递归函数，并在**main**函数中测试输入 输出。*

(a) 递归的定义：通过函数调用自身来解决问题

核心部分：

1. 基准条件：定义递归终止的条件 （通常是问题的最小规模或最简单情况，可以直接得出结果）
2. 递归条件：将原问题分解为更小的子问题，并通过调用自身来解决子问题

(b)

题目分析

基准条件：

`ClimbStairs(1) = 1` (只有一种方法)

`ClimbStairs(2) = 2` (2种方法，1+1 / 2)

递归条件：

`ClimbStairs(n) = ClimbStairs(n-1) + ClimbStairs(n-2) (n>2)`

```C
ClimbStairs(n) =
{
    1, n = 1;
    2, n = 2;
    ClimbStairs(n-1) + ClimbStairs(n-2), n>2;
}
```

(c)

```c
#include <stdio.h>
#include <stdlib.h>

int ClimbStairs(int n);

//测试函数
int main(void)
{
    int n;
    printf("请输入楼梯阶数n:");
    scanf("%d",&n);
    if (n<=0)
    {
        printf("error");
        return 1;
    }
    int result = ClimbStairs(n);
    printf("爬到%d阶楼梯有%d种不同的方法\n",n,result);
    return 0;
}

//递归函数实现
int ClimbStairs(int n)
{
    //基准条件
    if (n == 1) return 1;
    if (n == 2) return 2;

    //递归条件
    return ClimbStairs(n-1) + ClimbStairs(n-2);
}
```

时间复杂度 $O(2^n)$

Note

感觉用递归写比较复杂,减少重复计算的子问题
还有时间复杂度更低的 $O(N)$

> 斐波那契数列：prev1 前前项的值 / prev2 前一项的值 / current 当前值的值

```c
#include <stdio.h>
#include <stdlib.h>

int ClimbStairs(int n)
{
   if (n<=2) return n; //基准条件
   int prev1 = 1;
   int prev2 = 2;
   int current;
   for (int i=3;i<=n;i++)
   {
       current = prev1 + prev2;
       prev1 = prev2;
       prev2 = current;
   }
   return current;
}
```